

Core Language Working Group "ready" Issues for the February, 2016 (Jacksonville) meeting

Section references in this document reflect the section numbering of document [WG21 N4567](#).

1734. Nontrivial deleted copy functions

Section: 12.8 [class.copy] **Status:** ready **Submitter:** James Widman **Date:** 2013-08-09

The intent was for PODs in C++11 to be a superset of C++03 PODs. Consequently, in the following example, `c` should be a POD but isn't:

```
struct A {
    const int m;
    A& operator=(A const&) = default; // deleted and trivial, so A is a
                                     // POD, as it would be in 2003
                                     // without this explicit op= decl
};
static_assert(__is_trivially_copyable(A), "");

struct B {
    int i;
    B& operator=(B &) & = default;    // non-trivial
    B& operator=(B const&) & = default; // trivial
};

struct C {
    const B m;
    C& operator=(C const& r) = default; // deleted (apparently), but non-trivial (apparently)
    /* Notionally:
    C& operator=(C const& r) {
        (*this).m.operator=(r.m);
        return *this;
    }
    */
};
static_assert(!__is_trivially_copyable(C), "");
```

This is because of the following text from 12.8 [class.copy] paragraph 25:

for each non-static data member of `x` that is of class type (or array thereof), the assignment operator selected to copy/move that member is trivial;

In this case, overload resolution fails, so no assignment operator is selected, so `C::operator=(const C&)` is non-trivial.

(See also issue 1928.)

Additional note, November, 2014:

See paper N4148.

Additional note, October, 2015:

Moved from "extension" to "open" status, along with issue 1928, to allow reconsideration by CWG. It has been suggested that the triviality of a deleted function should be irrelevant, since it cannot be used in any event. A possible change to implement that,

more conservative than the one proposed in N4148, would be:

A trivially copyable class is a class that:

- has no non-trivial, non-deleted copy constructors (12.8 [class.copy]),
- has no non-trivial, non-deleted move constructors (12.8 [class.copy]),
- has no non-trivial, non-deleted copy assignment operators (13.5.3 [over.ass], 12.8 [class.copy]),
- has no non-trivial, non-deleted move assignment operators (13.5.3 [over.ass], 12.8 [class.copy]),
- has at least one non-deleted copy or move constructor or assignment operator, and
- has a trivial, non-deleted destructor (12.4 [class.dtor]).

Proposed resolution (October, 2015):

Change 9 [class] paragraph 6 as follows:

A *trivially copyable class* is a class ~~that~~:

- ~~has no non-trivial copy constructors (12.8 [class.copy]),~~
- ~~has no non-trivial move constructors (12.8 [class.copy]),~~
- ~~has no non-trivial copy assignment operators (13.5.3 [over.ass], 12.8 [class.copy]),~~
- ~~has no non-trivial move assignment operators (13.5.3 [over.ass], 12.8 [class.copy]), and~~
- **where each copy constructor, move constructor, copy assignment operator, and move assignment operator (12.8 [class.copy], 13.5.3 [over.ass]) is either deleted or trivial,**
- **that has at least one non-deleted copy constructor, move constructor, copy assignment operator, or move assignment operator, and**
- **that** has a trivial, **non-deleted** destructor (12.4 [class.dtor]).

1895. Deleted conversions in conditional operator operands

Section: 5.16 [expr.cond] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-03-17

In an example like,

```
struct B;
struct A { A(); A(B&) = delete; operator B&(); };
struct B : A { } b;
B &c = true ? A() : b;
```

the rules of 5.16 [expr.cond] paragraph 3 make this ambiguous: `A()` can be implicitly converted to the type “lvalue reference to `B`,” and `b` satisfies the constraints to be converted to an `A` prvalue (it’s of a type derived from `A` and the cv-qualifiers are okay). Bullet 3 bullet 1 is clear that we do not actually try to create an `A` temporary from `b`, so we don’t notice that it invokes a deleted constructor and rule out that conversion.

If the deleted conversion is in the other sense, the result is unambiguous:

```
struct B;
struct A { A(); A(B&); operator B&() = delete; };
struct B : A { } b;
B &c = true ? A() : b;
```

`A()` can no longer be implicitly converted to the type “lvalue reference to `B`”: since the declaration `B &t = A();` is not well formed (it invokes a deleted function), there is no implicit conversion. So we unambiguously convert the third operand to an `A` prvalue.

These should presumably either both be valid or both invalid. EDG and gcc call both ambiguous.

Notes from the June, 2014 meeting:

The wording should be changed to handle the convertibility test more like overload resolution: the conversion "exists" if the conversion function is declared, but is ill-formed if it would actually be used.

Proposed resolution (October, 2015):

1. Add the following as a new paragraph following 5.16 [expr.cond] paragraph 2:

Otherwise, if the second and third operand are glvalue bit-fields of the same value category and of types $cv1 \tau$ and $cv2 \tau$, respectively, the operands are considered to be of type $cv \tau$ for the remainder of this section, where cv is the union of $cv1$ and $cv2$.

2. Change 5.16 [expr.cond] paragraph 3 as follows:

Otherwise, if the second and third operand have different types and either has (possibly cv-qualified) class type, or if both are glvalues of the same value category and the same type except for cv-qualification, an attempt is made to **convert form an implicit conversion sequence (13.3.3.1 [over.best.ics]) from** each of those operands to the type of the other. **[Note: Properties such as access, whether an operand is a bit-field, or whether a conversion function is deleted are ignored for that determination. —end note]** ~~The process for determining whether an operand expression $E1$ of type $\tau1$ can be converted to match an operand expression $E2$ of type $\tau2$ is defined as follows:~~ **Attempts are made to form an implicit conversion sequence from an operand expression $E1$ of type $\tau1$ to a target type related to the type $\tau2$ of the operand expression $E2$ as follows:**

- If $E2$ is an lvalue: ~~$E1$ can be converted to match $E2$ if $E1$ can be implicitly converted (Clause 4 [conv]) to the type, the target type is~~ "lvalue reference to $\tau2$ ", subject to the constraint that in the conversion the reference must bind directly (8.5.3 [dcl.init.ref]) to an lvalue.
- If $E2$ is an xvalue: ~~$E1$ can be converted to match $E2$ if $E1$ can be implicitly converted to the type, the target type is~~ "rvalue reference to $\tau2$ ", subject to the constraint that the reference must bind directly.
- If $E2$ is a prvalue or if neither of the ~~conversions~~ **conversion sequences** above can be ~~done~~ **formed** and at least one of the operands has (possibly cv-qualified) class type:
 - ~~if $E1$ and $E2$ have class type, and the underlying class types are the same or one is a base class of the other: $E1$ can be converted to match $E2$ if the class of $\tau2$ is the same type as, or a base class of, the class of $\tau1$, and the cv-qualification of $\tau2$ is the same cv-qualification as, or a greater cv-qualification than, the cv-qualification of $\tau1$. If the conversion is applied, $E1$ is changed to a prvalue of type $\tau2$ by copy-initializing a temporary of type $\tau2$ from $E1$ and using that temporary as the converted operand. if $\tau1$ and $\tau2$ are the same class type (ignoring cv-qualification), or one is a base class of the other, and $\tau2$ is at least as cv-qualified as $\tau1$, the target type is $\tau2$,~~
 - ~~Otherwise (if $E1$ or $E2$ has a non-class type, or if they both have class types but the underlying classes are not the same and neither is a base class of the other): $E1$ can be converted to match $E2$ if $E1$ can be implicitly converted to~~ **otherwise, the target type is** the type that $E2$ would have after applying the lvalue-to-rvalue (4.1 [conv.lval]), array-to-pointer (4.2 [conv.array]), and function-to-pointer (4.3 [conv.func]) standard conversions.

Using this process, it is determined whether **an implicit conversion sequence can be formed from** the second operand ~~can be converted to match~~ **to the target type determined for** the third operand, and ~~whether the third operand can be converted to match the second operand~~ **vice versa**. If both ~~can be converted~~ **sequences can be formed**, or one ~~can be converted but the conversion is~~ **formed, but it is the** ambiguous **conversion sequence**, the program is ill-formed. If ~~neither can be converted~~ **no conversion sequence can be formed**, the operands are left unchanged and further checking is performed as described below. ~~If exactly one conversion is possible,~~ **Otherwise, if exactly one conversion sequence can be formed**, that conversion is applied to the chosen operand and the converted operand is used in place of the original operand for the remainder of this section. **[Note: The conversion might be ill-formed even if an implicit conversion sequence could be formed. —end note]**

This resolution also resolves issue 1932.

According to 7.1.1 [dcl.stc] paragraph 1,

If a *storage-class-specifier* appears in a *decl-specifier-seq*, there can be no `typedef` specifier in the same *decl-specifier-seq* and the *init-declarator-list* of the declaration shall not be empty...

This obviously should apply to `mutable` but does not because `mutable` applies to *member-declarator-lists*, not *init-declarator-lists*. Similarly, in 7.1.6.1 [dcl.type.cv] paragraph 1,

If a *cv-qualifier* appears in a *decl-specifier-seq*, the *init-declarator-list* of the declaration shall not be empty.

this should apply to member declarations as well.

Proposed resolution (October, 2015):

1. Change 7.1.1 [dcl.stc] paragraph 1 as follows:

...If a *storage-class-specifier* appears in a *decl-specifier-seq*, there can be no `typedef` specifier in the same *decl-specifier-seq* and the *init-declarator-list* **or member-declarator-list** of the declaration shall not be empty (except for an anonymous union declared in a named namespace or in the global namespace, which shall be declared `static` (9.5 [class.union])). The *storage-class-specifier* applies...

2. Change 7.1.6.1 [dcl.type.cv] paragraph 1 as follows:

...If a *cv-qualifier* appears in a *decl-specifier-seq*, the *init-declarator-list* **or member-declarator-list** of the declaration shall not be empty. [Note:...

Additional note, November, 2014:

The preceding resolution, which was advanced to "tentatively ready" status during the review session following the November, 2014 (Urbana) meeting, introduces an apparently unintentional conflict with 9.5 [class.union] paragraph 6 regarding the requirements for anonymous unions in unnamed namespaces and has been returned to "review" status to allow further discussion.

Notes from the October, 2015 meeting:

The proposed resolution was changed to address the preceding concern.

1932. Bit-field results of conditional operators

Section: 5.16 [expr.cond] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-02-21

According to 5.16 [expr.cond] paragraph 3,

if the second and third operand have different types and either has (possibly *cv-qualified*) class type, or if both are *glvalues* of the same value category and the same type except for *cv-qualification*, an attempt is made to convert each of those operands to the type of the other. The process for determining whether an operand expression E_1 of type T_1 can be converted to match an operand expression E_2 of type T_2 is defined as follows:

- If E_2 is an *lvalue*: E_1 can be converted to match E_2 if E_1 can be implicitly converted (Clause 4 [conv]) to the type "lvalue reference to T_2 ", subject to the constraint that in the conversion the reference must bind directly (8.5.3 [dcl.init.ref]) to an *lvalue*.

If two bit-field *glvalues* have exactly the same scalar type, paragraph 3 does not apply (two non-class operands must differ in at least *cv-qualification*). For an example like

```
struct S {
    int i:3;
    const int j:4;
} s;
int k = true ? s.i : s.j;
```

the condition is satisfied. The intent is that `S::i` can be converted to `const int` but `S::j` cannot be converted to `int`, so the result should be a bit-field *lvalue* of type `const int`. However, the test for convertibility is phrased in terms of direct reference binding, which is inapplicable to bit-fields, resulting in neither conversion succeeding, leading to categorizing the expression as ambiguous.

Proposed resolution (October, 2015):

This issue is resolved by the resolution of issue 1895.

1955. `#elif` with invalid controlling expression

Section: 16.1 [cpp.cond] **Status:** ready **Submitter:** Jonathan Wakely **Date:** 2014-06-25

An `#elif` is treated differently from an `#else` followed immediately by an `#if`: assuming the preceding `#if`'s condition was true, the condition in the second `#if` need not be a valid *expression*, while the condition in the `#elif` directive, though not evaluated, still must be syntactically correct.

[C DR 412](#) changes that for C; C++ should make the corresponding change.

Proposed resolution (November, 2014):

Change 16.1 [cpp.cond] paragraph 6 as follows:

Each directive's condition is checked in order. If it evaluates to false (zero), the group that it controls is skipped: directives are processed only through the name that determines the directive in order to keep track of the level of nested conditionals; the rest of the directives' preprocessing tokens are ignored, as are the other preprocessing tokens in the group. Only the first group whose control condition evaluates to true (nonzero) is processed; **any following groups are skipped and their controlling directives are processed as if they were in a group that is skipped**. If none of the conditions...

2001. *non-directive* is underspecified

Section: 16 [cpp] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-09-10

The Standard needs to describe *non-directives* more fully, e.g., whether they are ill-formed, conditionally-supported, etc. Since they are, in fact, directives, a different name might also be in order.

Proposed resolution (October, 2015):

1. Change 16 [cpp] paragraph 1 as follows:

```
group-part:
    if-section
    control-line
    text-line
    # non-directive conditionally-supported-directive
...
non-directive conditionally-supported-directive:
    pp-tokens new-line
```

2. Change 16 [cpp] paragraph 2 as follows:

A text line shall not begin with a `#` preprocessing token. A ~~non-directive~~ **conditionally-supported-directive** shall not begin with any of the directive names appearing in the syntax. **A conditionally-supported-directive is conditionally supported with implementation-defined semantics.**

2008. Default *template-arguments* underspecified

Section: 14.3 [temp.arg] **Status:** ready **Submitter:** Andrew Sutton **Date:** 2014-09-23

Proposed resolution, October, 2015:

Add the following as a new paragraph after 14.3 [temp.arg] paragraph 7:

When the template in a *template-id* is an overloaded function template...

When a *simple-template-id* does not name a function, a default *template-argument* is implicitly instantiated (14.7.1 [temp.inst]) when the value of that default argument is needed. [Example:

```
template<typename T, typename U = int> struct S { };  
S<bool>* p; // the type of p is S<bool, int>*
```

The default argument for *u* is instantiated to form the type *S<bool, int>**. —end example]

Notes from the November, 2014 meeting:

The preceding was extracted from the wording of the *Concepts Lite* draft Technical Specification.

2017. Flowing off end is not equivalent to no-expression return

Section: 6.6.3 [stmt.return] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-10-06

According to 6.6.3 [stmt.return] paragraph 2,

Flowing off the end of a function is equivalent to a return with no value...

This is not correct, since a `return` with no value is ill-formed in a value-returning function but flowing off the end results in undefined behavior.

Proposed resolution (May, 2015): [SUPERSEDED]

Change 6.6.3 [stmt.return] paragraph 2 as follows:

...Flowing off the end of a **value-returning** function is **undefined behavior. Flowing off the end of any other function is** equivalent to a return with no value; ~~this results in undefined behavior in a value-returning function.~~

Additional notes, October, 2015:

There is similar wording in 15.3 [except.handle] paragraph 14. Also, it might be better to avoid the use of the word “value”, since it is currently not clearly defined.

Proposed resolution (October, 2015):

1. Change 3.6.1 [basic.start.main] paragraph 5 as follows:

A `return` statement in `main` has the effect of leaving the `main` function (destroying any objects with automatic storage duration) and calling `std::exit` with the return value as the argument. If control ~~reaches~~ **flows off** the end of **the compound-statement of** `main` ~~without encountering a return statement~~, the effect is ~~that of executing~~ **equivalent to a return with operand 0 (see also 15.3 [except.handle]).**

~~return 0;~~

2. Change 6.6.3 [stmt.return] paragraph 2 as follows:

...Flowing off the end of a function **with a void return type** is equivalent to a return with no ~~value; this results in~~ **operand. Otherwise, flowing off the end of a function other than main (3.6.1 [basic.start.main] results in undefined behavior.**

3. Change 15.3 [except.handle] paragraph 14 as follows:

The currently handled exception is rethrown if control reaches the end of a handler of the *function-try-block* of a constructor or destructor. Otherwise, ~~a function returns when control reaches the end of a handler for the~~ *function-try-block* (6.6.3 [stmt.return]). Flowing off the end of a *function-try-block* is equivalent to a ~~return with no value; this results in undefined behavior in a value-returning function (6.6.3 [stmt.return])~~ **flowing off the end of the compound-statement of a handler of a function-try-block is equivalent to flowing off the end of the compound-statement of that function (see 6.6.3 [stmt.return]).**

2082. Referring to parameters in unevaluated operands of default arguments

Section: 8.3.6 [dcl.fct.default] **Status:** ready **Submitter:** Faisal Vali **Date:** 2015-02-09

According to 8.3.6 [dcl.fct.default] paragraph 9,

A default argument is evaluated each time the function is called with no argument for the corresponding parameter. The order of evaluation of function arguments is unspecified. Consequently, parameters of a function shall not be used in a default argument, even if they are not evaluated.

This prohibits use of parameters in unevaluated operands, e.g.,

```
void foo(int a = decltype(a){});
```

This wording predates the concept of “unevaluated operands” (the phrase “not evaluated” refers to calls to the function where an actual argument is supplied and thus the default argument is not used, not to unevaluated operands) and should not apply to such cases.

Proposed resolution (October, 2015):

1. Change 8.3.6 [dcl.fct.default] paragraph 7 as follows:

Local variables A local variable shall not be used appear as a potentially-evaluated expression in a default argument. [Example:

```
void f() {
    int i;
    extern void g(int x = i);           // error
    extern void h(int x = sizeof(i));   // OK
    // ...
}
```

—end example]

2. Change 8.3.6 [dcl.fct.default] paragraph 8 as follows:

[Note: The keyword this shall may not be used appear in a default argument of a member function; see 5.1.1 [expr.prim.general]. [Example:

```
class A {
    void f(A* p = this) { } // error
};
```

—end example] —end note]

3. Change 8.3.6 [dcl.fct.default] paragraph 9 as follows:

A default argument is evaluated each time the function is called with no argument for the corresponding parameter. The order of evaluation of function arguments is unspecified. Consequently, parameters of a function shall not be used in a default argument, even if they are not evaluated. A parameter shall not appear as a potentially-evaluated expression in a default argument. Parameters of a function declared before a default argument are in scope and can hide namespace and class member names. [Example:

```
int a;
int f(int a, int b = a);           // error: parameter a
                                   // used as default argument

typedef int I;
int g(float I, int b = I(2));      // error: parameter I found
int h(int a, int b = sizeof(a));   // error, parameter a used OK, unevaluated operand
                                   // in default argument
```

—end example] Similarly, a non-static member shall not be used appear in a default argument, even if it is not evaluated, unless it appears as the id-expression of a class member access expression (5.2.5 [expr.ref]) or unless it is used to form a pointer to member (5.3.1 [expr.unary.op]). [Example:...

2084. NSDMIs and deleted union default constructors

Section: 12.1 [class.ctor] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2015-02-12

According to 12.1 [class.ctor] paragraph 4 says,

A defaulted default constructor for class x is defined as deleted if:

- x is a union-like class that has a variant member with a non-trivial default constructor,
- ...

This should make the following example ill-formed:

```
struct S {  
    S();  
};  
union U {  
    S s{};  
} u;
```

because the default constructor of u is deleted. However, both clang and g++ accept this without error. Should the rule be relaxed for a union with an NSDMI?

Notes from the May, 2015 meeting:

An NSDMI is basically syntactic sugar for a *mem-initializer*, so the presence of one should be treated as if a user-declared default constructor were present.

Proposed resolution (October, 2015):

Change 12.1 [class.ctor] paragraph 4 as follows:

...A defaulted default constructor for class x is defined as deleted if:

- x is a union that has a variant member with a non-trivial default constructor and no variant member of x has a default member initializer,
- x is a union-like non-union class that has a variant member m with a non-trivial default constructor and no variant member of the anonymous union containing m has a default member initializer,
- ...

2093. Qualification conversion for pointer-to-member handler matching

Section: 15.3 [except.handle] **Status:** ready **Submitter:** David Majnemer **Date:** 2015-03-06

The criteria for matching an exception handler in 15.3 [except.handle] paragraph 3 include a qualification conversion for a handler of pointer or reference to pointer type but not for a handler of pointer-to-member type. However, current implementations permit such conversions.

Proposed resolution (October, 2015):

Change 15.3 [except.handle] bullet 3.3 as follows:

A *handler* is a match for an exception object of type E if

- ...
- the *handler* is of type $cv\ T$ or $const\ T\&$ where T is a pointer or pointer to member type and E is a pointer or pointer to member type that can be converted to T by either or both of
 - a standard pointer conversion (4.10 [conv.ptr]) not involving conversions to pointers to private or protected or ambiguous classes
 - a qualification conversion (4.4 [conv.qual]), or

- the *handler* is...

Notes from the October, 2015 meeting:

This resolution should not be adopted as a Defect Report, only a change in the working paper for future revisions of the Standard, because it could silently change the behavior of well-defined programs in implementations that conform to the existing wording.

2099. Inferring the bound of an array static data member

Section: 8.3.4 [dcl.array] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-03-15

According to 8.3.4 [dcl.array] paragraph 3,

An array bound may also be omitted when the declarator is followed by an *initializer* (8.5 [dcl.init]). In this case the bound is calculated from the number of initial elements...

However, the grammar for *member-declarator* uses *brace-or-equal-initializer*, not *initializer*, so the following is ill-formed:

```
struct X {  
    static constexpr int arr[] = { 1, 2, 3 };  
};
```

Proposed resolution (October, 2015):

Change 8.3.4 [dcl.array] paragraph 3 as follows:

...An array bound may also be omitted when the declarator is followed by an *initializer* (8.5 [dcl.init]) **or when a declarator for a static data member is followed by a brace-or-equal-initializer (9.2 [class.mem])**. In **this case both cases** the bound is calculated from the number of initial elements...

2124. Signature of constructor template

Section: 1.3.22 [defns.signature.member.templ] **Status:** ready **Submitter:** Hubert Tong **Date:** 2015-05-05

According to 1.3.22 [defns.signature.member.templ], the signature of a class member function template includes:

name, parameter type list (8.3.5 [dcl.fct]), class of which the function is a member, *cv-qualifiers* (if any), *ref-qualifier* (if any), return type, and template parameter list

However, a constructor template does not have a return type. This may be relevant to friend declaration matching.

Proposed resolution (October, 2015):

Change 1.3.22 [defns.signature.member.templ] as follows:

signature

<class member function template> name, parameter type list (8.3.5 [dcl.fct]), class of which the function is a member, *cv-qualifiers* (if any), *ref-qualifier* (if any), return type **(if any)**, and template parameter list

2130. Over-aligned types in *new-expressions*

Section: 5.3.4 [expr.new] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-05-28

According to 5.3.4 [expr.new] paragraph 1,

It is implementation-defined whether over-aligned types are supported (3.11 [basic.align]).

However, there is no mechanism for informing an allocation function of the required alignment for over-aligned types.

Nevertheless, 3.11 [basic.align] paragraph 9 says:

Additionally, a request for runtime allocation of dynamic storage for which the requested alignment cannot be honored shall be treated as an allocation failure.

This seems contradictory.

Proposed resolution (October, 2015):

Change 3.11 [basic.align] paragraph 9 as follows:

If a request for a specific extended alignment in a specific context is not supported by an implementation, the program is ill-formed. ~~Additionally, a request for runtime allocation of dynamic storage for which the requested alignment cannot be honored shall be treated as an allocation failure.~~

2157. Further disambiguation of enumeration *elaborated-type-specifier*

Section: 7.1.6.3 [dcl.type.elab] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-07-06

The resolution of issue 1966 does not apply to the case where the enumeration name is qualified, e.g.,

```
enum E : int;
struct X {
    enum ::E : int();
};
```

Proposed resolution (October, 2015):

Change 7.2 [dcl.enum] paragraph 1 as follows:

...A : following “enum ***nested-name-specifier_{opt}*** *identifier*” within the *decl-specifier-seq* of a *member-declaration* is parsed as part of an *enum-base*. [Note: This resolves a potential ambiguity...